

```

        if (userName.length() == 1) {
            maskingValue = "*";
        } else if (userName.length() == 2) {
            maskingValue = "*" + userName.substring(1);
        } else if (userName.length() == 3) {
            maskingValue = userName.substring(0, 1) + "*" + userName.substring(2);
        } else {
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < userName.length() - 2; i++) {
                sb.append("*");
            }
            maskingValue = userName.substring(0, 1) + sb.toString() +
            userName.substring(userName.length() - 1);
        }
    } catch (Exception e) {
        if (LOGGER.isWarnEnabled()) {
            LOGGER.warn("elkFormatUserNameWithAsterisk ERROR : [{}]", userName);
        }
    }
    return maskingValue;
}

/**
 * Exception 정보 간소화 조회
 * @param e
 * @return
 */
public static String getSimpleExceptionInfo(Throwable e) {
    String exceptionInfo = "";
    try {
        if (ObjectUtils.isEmpty(e)) {
            StackTraceElement[] se = e.getStackTrace();
            String throwableExceptionClass = e.getClass().toString();
            String exceptionClassName = se[0].getClassName();
            String exceptionMethodName = se[0].getMethodName();
            int exceptionThrowableLine = se[0].getLineNumber();

            exceptionInfo = throwableExceptionClass + ": " + exceptionClassName + "." +
            exceptionMethodName + ":" + exceptionThrowableLine;
        }
    } catch (Exception ex) {
        LOGGER.error("CommonUtil > getExceptionInfo exception.");
        LOGGER.error("{} ", ex.getMessage());
        exceptionInfo = e.getMessage();
    }

    return exceptionInfo;
}

}

package nhnis.fw.common.util;

import java.sql.Timestamp;
import java.text.DateFormat;
import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Date;
import java.util.Locale;
import java.util.TimeZone;

public class DateUtil {
    private static SimpleDateFormat dateFormatter = new SimpleDateFormat("yyyyMMdd",

```

```

Locale.KOREA);

    public static String[] dateFormatterList = new String[] { "yyyy-MM-dd",
"yyyy/MM/dd", "yyyyMMdd", "yyyy년 MM월 dd일", "yyyy.MM.dd", "yyyy MM dd" };

    private static SimpleDateFormat timeFormatter = new SimpleDateFormat("HHmmss",
Locale.KOREA);

    public static String[] formatListTime = new String[] {
        "HH:mm:ss", "HH:mm", "HH", "a hh:mm:ss", "a hh:mm", "a hh", "HHmmss",
"yyyyMMddHHmmSSS", "yyyy-MM-dd HH:mm:ss", "yyyyMMdd",
        "HHmmSSS" };

    private static SimpleDateFormat militimeFormatter = new
SimpleDateFormat("HHmmssSSS", Locale.KOREA);

    private static SimpleDateFormat dateTimeFormatter = new
SimpleDateFormat("yyyyMMddHHmmss", Locale.KOREA);

    private static TimeZone tz = TimeZone.getTimeZone("Asia/Seoul");

    static {
        dateFormatter.setTimeZone(tz);
        timeFormatter.setTimeZone(tz);
        militimeFormatter.setTimeZone(tz);
        dateTimeFormatter.setTimeZone(tz);
    }

    public static Date getDate() {
        return getDate();
    }

    public static Timestamp getTimestamp() {
        return new Timestamp(getSystemDate().getTime());
    }

    public static String getCurrentTime() {
        SimpleDateFormat fmt = new SimpleDateFormat("yyyyMMddHHmmss");
        String time = fmt.format(new Date(System.currentTimeMillis()));
        return time;
    }

    public static String getCurrentTime(String format) {
        SimpleDateFormat fmt = new SimpleDateFormat(format);
        String time = fmt.format(new Date(System.currentTimeMillis()));
        return time;
    }

    public static String getCurrentTime(int format) {
        if (format >= formatListTime.length || format < 0)
            return null;
        SimpleDateFormat simpledateformat = new
SimpleDateFormat(formatListTime[format]);
        Calendar calendar = Calendar.getInstance();
        simpledateformat.setCalendar(calendar);
        String s =
simpledateformat.format(simpledateformat.getCalendar().getTime());
        return s;
    }

    public static Date toDate(String strDate) throws IllegalArgumentException {
        return (new CalendarWrapper(strDate)).getDate();
    }

```

```

}

public static String toDateString() {
    synchronized (dateFormatter) {
        return dateFormatter.format(getSystemDate());
    }
}

public static String toDateString(Date date) {
    if (date == null || date.toString().startsWith("1900"))
        return "";
    synchronized (dateFormatter) {
        return dateFormatter.format(date);
    }
}

public static String toDateString(Timestamp date) {
    if (date == null || date.toString().startsWith("1900"))
        return "";
    synchronized (dateFormatter) {
        return dateFormatter.format(date);
    }
}

public static String toTimeString() {
    synchronized (timeFormatter) {
        return timeFormatter.format(getSystemDate());
    }
}

public static String toTimeString(Date date) {
    if (date == null || date.toString().startsWith("1900"))
        return "";
    synchronized (timeFormatter) {
        return timeFormatter.format(date);
    }
}

public static String toTimeString(Timestamp date) {
    if (date == null || date.toString().startsWith("1900"))
        return "";
    synchronized (timeFormatter) {
        return timeFormatter.format(date);
    }
}

public static String toMilliTimeString() {
    synchronized (millitimeFormatter) {
        return millitimeFormatter.format(getSystemDate());
    }
}

public static String toDateTimeString(String format) {
    DateFormat df = new SimpleDateFormat(format);
    df.setTimeZone(tz);
    return df.format(getSystemDate());
}

public static String toDateTimeString() {
    synchronized (dateTimeFormatter) {
        return dateTimeFormatter.format(getSystemDate());
    }
}

```

```

    }

    public static String toDateTimeString(Date date) {
        if (date == null || date.toString().startsWith("1900"))
            return "";
        synchronized (dateFormatter) {
            return dateTimeFormatter.format(date);
        }
    }

    public static String toDateTimeString(Timestamp date) {
        if (date == null || date.toString().startsWith("1900"))
            return "";
        synchronized (dateTimeFormatter) {
            return dateTimeFormatter.format(date);
        }
    }

    public static Timestamp toTimestamp(String strDate) throws IllegalArgumentException
    {
        return new Timestamp((new CalendarWrapper(strDate)).getDate().getTime());
    }

    public static Timestamp toTimestamp(Date date) {
        if (date == null || date.toString().startsWith("1900"))
            return null;
        return new Timestamp(date.getTime());
    }

    public static boolean isValidDate(String strDate) {
        if (strDate == null)
            return false;
        String tempStrDate = strDate.replaceAll("\\D", "");
        try {
            CalendarWrapper calendar = new CalendarWrapper(tempStrDate);
            int len = tempStrDate.length();
            if (len >= 4 && calendar.getYear() !=
Integer.parseInt(tempStrDate.substring(0, 4)))
                return false;
            if (len >= 6 && calendar.getMonth() !=
Integer.parseInt(tempStrDate.substring(4, 6)))
                return false;
            if (len >= 8 && calendar.getDay() !=
Integer.parseInt(tempStrDate.substring(6, 8)))
                return false;
            if (len >= 10 && calendar.getHour() !=
Integer.parseInt(tempStrDate.substring(8, 10)))
                return false;
            if (len >= 12 && calendar.getMinute() !=
Integer.parseInt(tempStrDate.substring(10, 12)))
                return false;
            if (len >= 14 && calendar.getSecond() !=
Integer.parseInt(tempStrDate.substring(12, 14)))
                return false;
            return !(len >= 17 && calendar.getMilliSecond() !=
Integer.parseInt(tempStrDate.substring(14, 17)));
        } catch (IllegalArgumentException e) {
            return false;
        }
    }

    public static Date getBeforeDate(Date date, int year, int month, int day) {

```

```

        return (new CalendarWrapper(date)).beforeDate(year, month, day);
    }

    public static Date getBeforeDate(String stdDate, int year, int month, int day)
throws IllegalArgumentException {
        return (new CalendarWrapper(stdDate)).beforeDate(year, month, day);
    }

    public static String getBeforeDateString(String stdDate, int year, int month, int
day) throws IllegalArgumentException {
        return (new CalendarWrapper((new CalendarWrapper(stdDate)).beforeDate(year,
month, day))).getDateString();
    }

    public static Date getBeforeDate(String stdDate, int year, int month, int day, int
hour, int min, int sec) throws IllegalArgumentException {
        return (new CalendarWrapper(stdDate)).beforeDate(year, month, day, hour,
min, sec);
    }

    public static String getBeforeDateString(String stdDate, int year, int month, int
day, int hour, int min, int sec) throws IllegalArgumentException {
        return (new CalendarWrapper((new CalendarWrapper(stdDate)).beforeDate(year,
month, day, hour, min, sec))).getTimeString();
    }

    public static Date getAfterDate(Date date, int year, int month, int day) {
        return (new CalendarWrapper(date)).afterDate(year, month, day);
    }

    public static Date getAfterDate(String stdDate, int year, int month, int day)
throws IllegalArgumentException {
        return (new CalendarWrapper(stdDate)).afterDate(year, month, day);
    }

    public static String getAfterDateString(String stdDate, int year, int month, int
day) throws IllegalArgumentException {
        return (new CalendarWrapper((new CalendarWrapper(stdDate)).afterDate(year,
month, day))).getDateString();
    }

    public static Date getAfterDate(String stdDate, int year, int month, int day, int
hour, int min, int sec) throws IllegalArgumentException {
        return (new CalendarWrapper(stdDate)).afterDate(year, month, day, hour, min,
sec);
    }

    public static String getAfterDateString(String stdDate, int year, int month, int
day, int hour, int min, int sec) throws IllegalArgumentException {
        return (new CalendarWrapper((new CalendarWrapper(stdDate)).afterDate(year,
month, day, hour, min, sec))).getTimeString();
    }

    public static long getDaysDiff(String beginDate, String endDate) throws
IllegalArgumentException {
        return getDaysDiff(toDate(beginDate), toDate(endDate));
    }

    private static long getDaysDiff(Date beginDate, Date endDate) {
        if (endDate.before(beginDate))
            return -doGetDaysDiff(endDate, beginDate);
        return doGetDaysDiff(beginDate, endDate);
    }

```

```

    }

    private static long doGetDaysDiff(Date beginDate, Date endDate) {
        long diff = endDate.getTime() - beginDate.getTime();
        return diff / 86400000L;
    }

    public static int getMonthsDiff(String beginDate, String endDate) throws
    IllegalArgumentException {
        return getMonthsDiff(toDate(beginDate), toDate(endDate));
    }

    public static int getMonthsDiff(Date beginDate, Date endDate) {
        if (endDate.before(beginDate))
            return -doGetMonthsDiff(endDate, beginDate);
        return doGetMonthsDiff(beginDate, endDate);
    }

    private static int doGetMonthsDiff(Date beginDate, Date endDate) {
        CalendarWrapper begin = new CalendarWrapper(beginDate);
        CalendarWrapper end = new CalendarWrapper(endDate);
        int diffMonth = (end.getYear() - begin.getYear()) * 12 + end.getMonth() -
        begin.getMonth();
        int diffDays = end.getDay() - begin.getDay();
        if (diffMonth >= 0) {
            if (diffDays < 0)
                diffMonth--;
        } else if (diffDays > 0) {
            diffMonth++;
        }
        return diffMonth;
    }

    public static int getYearsDiff(String beginDate, String endDate) throws
    IllegalArgumentException {
        return getYearsDiff(toDate(beginDate), toDate(endDate));
    }

    public static int getYearsDiff(Date beginDate, Date endDate) {
        if (endDate.after(beginDate))
            return doGetYearsDiff(beginDate, endDate);
        return -doGetYearsDiff(endDate, beginDate);
    }

    private static int doGetYearsDiff(Date beginDate, Date endDate) {
        return getMonthsDiff(beginDate, endDate) / 12;
    }

    public static long getDaysDiffAbove(String beginDate, String endDate) throws
    IllegalArgumentException {
        return getDaysDiffAbove(toDate(beginDate), toDate(endDate));
    }

    public static long getDaysDiffAbove(Date beginDate, Date endDate) {
        if (endDate.before(beginDate))
            return -doGetDaysDiffAbove(endDate, beginDate);
        return doGetDaysDiffAbove(beginDate, endDate);
    }

    private static long doGetDaysDiffAbove(Date beginDate, Date endDate) {
        long diff = endDate.getTime() - beginDate.getTime();
        double dayDiff = diff / 8.64E7D;
    }

```